Keep Setup and Test Code Separate



Consider the traditional three steps of testing: *arrange*, *act*, and *assert*. Moving just the arrange (setup) step into a `before` hook makes it clear what is and isn't part of the behavior you're testing. You still retain the flexibility to add additional setup code for individual specs that need it.

Handling Failure

Now that your specs are testing the "happy path" of a successful expense, let's turn our attention to the failure case. You can apply the hard-won knowledge from the previous sections, and start with an already-factored `before` hook. Here, the setup code will fill the `RecordResult` value object with a `false` success status and an error message:

05-unit-specs/06/expense_tracker/spec/unit/app/api_spec.rb

```
context 'when the expense fails validation' do
  let(:expense) { { 'some' => 'data' } }

  before do
    allow(ledger).to receive(:record)
      .with(expense)
      .and_return(RecordResult.new(false, 417, 'Expense incomplete'))
  end

  it 'returns an error message' do
    post '/expenses', JSON.generate(expense)

    parsed = JSON.parse(last_response.body)
    expect(parsed).to include('error' => 'Expense incomplete')
  end

  it 'responds with a 422 (Unprocessable entity)' do
    post '/expenses', JSON.generate(expense)
    expect(last_response.status).to eq(422)
  end
end
```

The expectations in the examples are different, too; they're checking for an error message in the body and an HTTP error code.

These new specs will fail when you run them, as the behavior isn't implemented yet:

```
$ bundle exec rspec spec/unit/app/api_spec.rb
« truncated »

Failures:

  1) ExpenseTracker::API POST /expenses when the expense fails validation ↩
  returns an error message
```